Department of Electronics & Communications

25SC2107E – Machine Learning

Skilling Experiment 4

California Housing — Linear Regression and Regularisation

Dataset: California Housing | Course Outcome: CO2 | Environment: VS Code

Prepared by: **Dr. Omprakash Gottam**, Dept. of ECE, KLEF Hyderabad

1 Experiment 4: California Housing — Linear Regression and Regularisation

Dataset: California Housing **Course Outcome:** CO2

1.1 Aim

To fit a linear regression model predicting California house prices, to reproduce the same fit by hand with NumPy gradient descent, and to compare Ridge, Lasso and ElasticNet regularisation across a range of penalty strengths.

1.2 Learning Objectives

After completing this experiment, students should be able to:

- Tell a *regression* problem apart from the classification of Experiments 1 to 3.
- Fit `LinearRegression` and measure it with RMSE and R^2.
- Write gradient descent in NumPy and check it against scikit-learn.
- Explain what Ridge, Lasso and ElasticNet punish, and what `alpha` controls.

1.3 Environment and Libraries

VS Code with the Python + Jupyter extensions, `.venv` activated.

```
pip install numpy pandas matplotlib scikit-learn
```

1.4 Dataset

California Housing, from the 1990 US census. Each row is one census *district*, not one house.

- 20,640 rows, 8 numeric features, nothing missing. There is no cleaning to do.
- **Answer column:** `MedHouseVal`, in hundreds of thousands of dollars, so 2.5 means $250,000.
- Downloaded by `fetch_california_housing` (about 400 kB, then cached, so later runs work offline).

Note

A different kind of answer

In Experiments 1 to 3 the answer was a **label**. Here it is a **number**, and that one change alters everything.

	Classification (Exp. 1–3)	Regression (this experiment)
The answer is	a category	a number
Model used	LogisticRegression	LinearRegression
Measured by	accuracy	RMSE and R^2
Baseline to beat	the commonest class	the average price

No predicted price is ever exactly right, so "how often is it correct?" is a pointless question. The useful one is *how far wrong is it, on average?*

1.5 Procedure

1. Create a notebook `skill_lab4.ipynb`; select the `.venv` interpreter.
2. Load California Housing and look at the answer column.
3. Split into training and test sets, then scale the features.

4. Fit `LinearRegression` and report RMSE and R^2.
5. Write the same fit by hand with NumPy and plot the loss curve.
6. Compare the two sets of coefficients.
7. Plot test RMSE against `alpha` for Ridge, then for Lasso.
8. Compare all three sets of coefficients on one bar chart.
9. Write `regression_benchmark()` to rank every model by test error.

1.6 Notebook Implementation

Cell 1 — Import the libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso,
    ElasticNet
from sklearn.metrics import mean_squared_error, r2_score
```

- The four models on the `linear_model` line are closer relatives than they look. **All four draw a straight line**, differing only in how strongly they are punished for using large coefficients.

Cell 2 — Load the data

```
data = fetch_california_housing(as_frame=True)
housing = data.frame
print(housing.shape)
housing.head()
```

- `as_frame=True` returns a pandas table so the columns keep their names. You need those names in Cell 10.
- `data.frame` holds the 8 features and the answer together: 20,640 rows by 9 columns.

Cell 3 — Look at the answer column

```
plt.hist(housing["MedHouseVal"], bins=50)
plt.title("Median House Value")
plt.show()
```

- The distribution is **right-skewed**, with a long thin tail of expensive districts.
- **Look at the far right: a tall narrow spike at exactly 5.0. That spike is not real, it is a cap.** Every district worth more than $500,000 was recorded as 5.0, so those districts are not all worth the same, only *at least* that much.
- Consequence: the model can never predict above 5.0, so the biggest errors will fall among the most expensive districts.

Cell 4 — Split into training and test sets

```
X = housing.drop(columns="MedHouseVal")
y = housing["MedHouseVal"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
print(X_train.shape, X_test.shape)
```

- 16,512 rows to train on, 4,128 held back.
- There is no `stratify` here: stratification balances *categories*, and a price is not a category.

Cell 5 — Scale the features

```
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
print(X_train_s.mean().round(3), X_train_s.std().round(3))
```

Fit on train, transform on both. Here scaling is **compulsory**, for two separate reasons:

- **Gradient descent (Cell 6) would break.** One step size serves all features at once. With `Population` in the thousands and `AveBedrms` near 1, no single step suits both.
- **Regularisation (Cell 8 onward) would be unfair.** Ridge and Lasso punish *large coefficients*. A feature measured in thousands has a naturally tiny coefficient and would escape the penalty for no good reason.

Cell 6 — Fit with scikit-learn, then by hand

```
lr = LinearRegression()
lr.fit(X_train_s, y_train)
pred = lr.predict(X_test_s)
print("RMSE:", round(np.sqrt(mean_squared_error(y_test, pred)), 4))
print("R2  :", round(r2_score(y_test, pred), 4))

w = np.zeros(X_train_s.shape[1])
b = 0.0
rate = 0.1
losses = []

for step in range(500):
    pred_train = X_train_s @ w + b
    error = pred_train - y_train.to_numpy()
    losses.append((error ** 2).mean())
    w = w - rate * (X_train_s.T @ error) / len(y_train)
    b = b - rate * error.mean()

print("Final MSE:", round(losses[-1], 4))
```

- **RMSE** ≈ 0.75: the model is typically off by roughly \$75,000. $R^2 \approx 0.58$: it explains a little under three fifths of the variation. R^2 can go *below zero*, meaning worse than always guessing the average, and you will see that in the deliverable.
- The loop below is the whole training algorithm that `lr.fit()` hid. `w` holds one weight per feature and `b` the intercept, both starting at zero. `X @ w + b` predicts all 16,512 rows at once.
- `rate` is the learning rate. Too small and 500 steps are not enough; too large and the loop overshoots to `NaN`.

Cell 7 — The loss curve, and did it match?

```
plt.plot(losses)
plt.xlabel("step")
plt.ylabel("mean squared error")
plt.title("Gradient Descent Loss")
plt.show()

compare = pd.DataFrame({
    "feature": X.columns,
```

```
    "gradient_descent": w.round(4),
    "sklearn": lr.coef_.round(4),
})
compare
```

- The curve should fall steeply then flatten. Flattening means **converged**. Still falling at step 500 means the rate is too small; rising or `NaN` means too large; flat from the start means the features were not scaled.
- The two coefficient columns agree to four decimal places. `LinearRegression` **is not magic** — you have rebuilt it in eleven lines. Scikit-learn solves the equations directly in one exact step; gradient descent walks downhill in many small ones, and keeps working when the data is too large to solve at once.
- `MedInc` carries by far the largest weight: district income dominates house price.

Cell 8 — Ridge: test error against alpha

```
alphas = [0.001, 0.01, 0.1, 1, 10, 100]
ridge_rmse = []

for a in alphas:
    model = Ridge(alpha=a)
    model.fit(X_train_s, y_train)
    ridge_rmse.append(np.sqrt(mean_squared_error(y_test,
        model.predict(X_test_s))))

plt.plot(alphas, ridge_rmse, marker="o")
plt.xscale("log")
plt.xlabel("alpha")
plt.ylabel("test RMSE")
plt.title("Ridge")
plt.show()
```

- **Ridge is linear regression plus a fine** on the *squared* size of each coefficient. `alpha` sets how heavy the fine is: at 0 it is ordinary regression, and as it grows all coefficients shrink towards zero but **none ever reaches zero**.
- `xscale("log")` is needed because the alphas span five powers of ten.
- The curve is almost flat. That is the correct result, not a failure: with 16,512 rows and only 8 features there is little overfitting to repair. **Reporting "it made no difference" is a finding.**

Cell 9 — Lasso: test error against alpha

```
lasso_rmse = []

for a in alphas:
    model = Lasso(alpha=a)
    model.fit(X_train_s, y_train)
    lasso_rmse.append(np.sqrt(mean_squared_error(y_test,
        model.predict(X_test_s))))

plt.plot(alphas, lasso_rmse, marker="o")
plt.xscale("log")
plt.xlabel("alpha")
plt.ylabel("test RMSE")
plt.title("Lasso")
plt.show()
```

- **Lasso fines the *plain* size of each coefficient, not the squared size.** That single

change lets a coefficient reach **exactly zero**, removing the feature entirely, so Lasso selects features while it fits.

- Ridge stayed flat; Lasso is flat then climbs steeply. By `alpha` = 1 every coefficient is zero, the model predicts one constant, and RMSE jumps to what guessing the average would give.
- **Too much regularisation does not merely fail to help. It destroys the model.**

Cell 10 — What the fines did to the coefficients

```
coefs = pd.DataFrame({
    "LinearRegression": lr.coef_,
    "Ridge a=1": Ridge(alpha=1).fit(X_train_s, y_train).coef_,
    "Lasso a=0.1": Lasso(alpha=0.1).fit(X_train_s, y_train).coef_,
}, index=X.columns)

coefs.plot.bar(figsize=(10, 4))
plt.ylabel("coefficient")
plt.title("Coefficients under each penalty")
plt.show()
```

- Ridge bars sit slightly inside the plain regression bars: shrunk, but all still present. Lasso bars are different in kind: several have vanished to exactly zero.
- **Ridge says use every feature, but gently. Lasso says use fewer features.**
- `MedInc` survives every fine. When a feature refuses to be regularised away, it is telling you something.

Deliverable Function

```
def regression_benchmark(X_train, y_train, X_test, y_test, alphas):
    models = {"LinearRegression": LinearRegression()}

    for a in alphas:
        models["Ridge a=" + str(a)] = Ridge(alpha=a)
        models["Lasso a=" + str(a)] = Lasso(alpha=a)
        models["ElasticNet a=" + str(a)] = ElasticNet(alpha=a,
            l1_ratio=0.5)

    rows = []
    for name in models:
        model = models[name]
        model.fit(X_train, y_train)
        pred = model.predict(X_test)
        rows.append({
            "model": name,
            "test_rmse": np.sqrt(mean_squared_error(y_test, pred)),
            "test_r2": r2_score(y_test, pred),
            "zero_coefs": int((model.coef_ == 0).sum()),
        })

    table = pd.DataFrame(rows)
    table = table.sort_values("test_rmse").reset_index(drop=True)
    return table.round(4)


results = regression_benchmark(X_train_s, y_train, X_test_s, y_test,
                               [0.01, 0.1, 1, 10])
results
```

- The pattern is worth keeping: build a dictionary of named models, loop over it, collect one

dictionary of results per model, turn the list into a DataFrame at the end. Adding a fifth model later costs one line.

- **ElasticNet** mixes the other two. `l1_ratio=0.5` means half Lasso, half Ridge, so it can delete a coefficient while still handling correlated features gracefully.
- **Read `zero_coefs` alongside RMSE**: it stays at 0 for every Ridge, rises for Lasso and ElasticNet as `alpha` grows, and reaches 8 at the bottom where every feature has gone. Those rows have $R^2 \approx 0$, the signature of a model fined into predicting one constant forever.
- Sorting by `test_rmse` puts the winner first. Expect the top rows to be nearly tied: on this dataset regularisation neither helps nor hurts much, and a table that shows the ties is more useful than one that hides them.

1.7 Expected Output

```
(20640, 9)
(16512, 8) (4128, 8)
RMSE: 0.7456
R2  : 0.5758
Final MSE: 0.5179

                model  test_rmse  test_r2  zero_coefs
0   LinearRegression      0.7456   0.5758           0
1       Ridge a=0.01      0.7456   0.5758           0
2        Ridge a=0.1      0.7456   0.5758           0
3          Ridge a=1      0.7456   0.5758           0
...
11         Lasso a=1      1.1453  -0.0001           8
12    ElasticNet a=10      1.1453  -0.0001           8
```

1.8 Result

Linear regression on the standardised California Housing features attained a test RMSE of about 0.75, roughly $75,000, and an R^2 of about 0.58. A NumPy implementation of gradient descent, run for 500 steps at a learning rate of 0.1, converged to coefficients agreeing with the closed-form solution to four decimal places. Ridge left the error essentially unchanged, while Lasso and ElasticNet progressively eliminated coefficients until all 8 were zero and the model degenerated to a constant predictor.

Note

The three fines, one line each

- **Ridge** fines *squared* coefficients: shrinks all, deletes none. Use it when every feature carries a little signal.
- **Lasso** fines *plain* coefficients: can set them to exactly zero. Use it when you suspect many features are useless and want the model to say which.
- **ElasticNet** mixes the two, controlled by `l1_ratio`. Use it when features move together *and* you want selection.

In all three, `alpha` is the volume knob. Turn it to zero and you have plain linear regression; turn it far enough and you have a model that predicts one number forever.